

Título de tu Informe

Tu Nombre

2 de octubre de 2026

1. Consideraciones

Este documento está pensado para que sirva de guía, e incluso template a la hora de realizar un informe para la materia (por no decir, cualquier documento formal en la facultad). La idea es utilizar $\text{\LaTeX}$, pura y exclusivamente para que tenga un formato acorde. Es decir, no hecho en un simple procesador de texto (como pudiese ser Word).

Aquí sólo se mostraran algunas de las cosas que se pueden hacer con $\text{\LaTeX}$. Si se quiere hacer algo específico, o de otra forma, abunda la documentación (y artículos de StackOverflow) al respecto.

Definimos ciertos lineamientos técnicos a considerar para los informes:

- En el informe debe estar incluido el código más importante (es decir, los algoritmos, no un main) cuando corresponda.
- Debe estar explicado cómo se hacen los sets de prueba. Inclusive, si notan alguna falencia con esta generación que pueda afectar de alguna forma el análisis del resultado, se espera que lo enuncien (hay trabajos donde esto no sucederá, pero hay trabajos donde sí, especialmente cuando vemos aproximaciones).
- Los gráficos deben estar bien generados: deben tener título, los ejes deben tener tanto qué refieren como qué unidad de medida usan (en caso de tenerla). En caso de tener gráficos superpuestos, debe quedar claro qué es cada gráfico.
- **Importante:** en caso de tener que realizar una reentrega (y por ende, tener que cambiar cosas del informe), salvo que haya que rehacer una enorme cantidad del mismo, no modificar el informe anterior sino agregar un anexo al final indicando las correcciones realizadas en la nueva entrega, para agilizar las correcciones.

A su vez, dejamos algunas breves líneas sobre cuestiones de redacción:

- Evitar errores ortográficos y gramaticales.
- El lenguaje debe ser técnico. No tiene que por eso ser un texto aburrido, pero evitar frases como "*la cosa que más me complicó fue...*", "*creo que...*".
- Usar plural de modestia, incluso si el trabajo fuera realizado por una única persona.

Resolución

2. Algoritmo para elegir las monedas

En este trabajo de ejemplo realizaremos el análisis teórico y empírico de diferentes algoritmos que sirven para resolver el mismo problema: obtener el orden en que se deben elegir las monedas para maximizar la ganancia obtenida de un arreglo de n monedas.

2.1. Algoritmo para encontrar el optimo

El algoritmo plantea una matriz de $n * n$, siendo n la cantidad de monedas, donde para cada coordenada i, j de la matriz, encontramos una solución para un problema de dimensiones i, j . Luego se llenan las diagonales, donde las coordenadas i y j son iguales, $i = j$, y la diagonal donde la coordenada j es uno más que la i , $i = j + 1$, con los valores de los dos casos base, siendo estos:

- tener una sola moneda, donde el óptimo es la moneda en sí.
- tener dos monedas, donde el óptimo es la más grande entre las dos.

Luego recorre la parte superior de la matriz (ya que los punteros i, j que apuntan al principio y final del array de monedas respectivamente, nunca se intercambian, lo que es igual, se cumple que $i \leq j$), llenando cada lugar de la matriz con la solución del problema de tamaño i, j . Es así que una vez completamos la matriz, encontramos la solución óptima para nuestro problema original en la coordenada $i = 0$ y $j = n$.

Aquí el código usado para generar la matriz con los valores óptimos para todos los problemas de dimensiones i, j .

```
1 def elegir(monedas):
2     opt = [[0] * len(monedas) for i in range(len(monedas))]
3
4     # Casos base con una moneda
5     for i in range(0, len(monedas)):
6         opt[i][i] = monedas[i]
7
8     # Casos base con 2 monedas
9     for i in range(0, len(monedas) - 1):
10         j = i + 1
11         opt[i][j] = max(monedas[i], monedas[j])
12
13     for i in range(len(opt) - 1, -1, -1):
14         for j in range(i + 2, len(opt)):
15             # Sophia toma la primera
16             # Achico mi rango según lo que elija Mateo
17             opt_i = opt[i + 2][j] if monedas[i + 1] > monedas[j]
18             else opt[i + 1][j - 1]
```

```

19         # Sophia toma la ultima
20         # Achico mi rango segun lo que eliga Mateo
21         opt_j = opt[i + 1][j - 1] if monedas[i] > monedas[j -
1] else opt[i][j - 2]
22
23         opt[i][j] = max(monedas[i] + opt_i, monedas[j] + opt_j)
24
25     maxima_ganancia = opt[0][len(opt) - 1]
26     sol = solucion(opt, monedas, [], 0, len(opt) - 1)
27     return sol

```

La forma de obtener nuestra matriz optima es la siguiente. Se plantea que para cada problema de dimensiones i, j , donde queremos encontrar que moneda elegir para maximizar la ganancia, la moneda en la posición i (la primera), y la moneda j (la ultima), se pueden analizar las siguientes opciones.

1. Sophia toma la primera moneda, la moneda i y Mateo elige la mayor entre las monedas $i + 1$ y j .
 - a) Si mateo elige la moneda $i + 1$ el rango (i, j) se achica a $(i + 2, j)$, por lo que nuestra solución será el valor de la moneda i mas la solución del rango $(i + 2, j)$.

$$opt(i, j) = moneda_i + opt(i + 2, j)$$
 - b) Si mateo elige la moneda j el rango (i, j) se achica a $(i + 1, j - 1)$, por lo que nuestra solución será el valor de la moneda i mas la solución del rango $(i + 1, j - j)$.

$$opt(i, j) = moneda_i + opt(i + 1, j - 1)$$
2. Sophia toma ahora la ultima moneda, la moneda j y Mateo elige la mayor entre las monedas i y $j - 1$.
 - a) Si mateo eligio la moneda i el rango (i, j) se achica a $(i + 1, j - 1)$, por lo que nuestra solución será el valor de la moneda j mas la solución del rango $(i + 1, j - 1)$.

$$opt(i, j) = moneda_j + opt(i + 1, j - 1)$$
 - b) Si mateo eligio la moneda j el rango (i, j) se achica a $(i, j - 2)$, por lo que nuestra solución será el valor de la moneda j mas la solución del rango $(i, j - 2)$.

$$opt(i, j) = moneda_j + opt(i, j - 2)$$

Luego de plantear las cuatro posibles situaciones, podemos ver que, sin importar lo que elija sophia, mateo siempre elige la moneda mas grande por lo que en ambos casos en simplemente realizar una comparacion para ver cual es mas grande, segun la situacion de monedas. Esta comparacion determina el rango de mis soluciones parciales. Luego obtengo mis soluciones parciales para los rangos respectivos segun la eleccion de mateo, me quedo con aquella solución, que maximize mi ganancia. Entonces Siendo opt_i , la solución para un rango (i_i, j_i) ,

asumiendo que sophia toma la *moneda_i*, y, siendo *opt_j*, la solución para un rango (*i_j*, *j_j*), asumiendo que sophia toma la moneda *j*, el óptimo para el rango (*i*, *j*) queda de la siguiente manera.

$$\text{opt}(i, j) = \max(\text{moneda}_i + \text{opt}_i, \text{moneda}_j + \text{opt}_j)$$

donde:

$$\text{moneda}_i + \text{opt}_i = \begin{cases} \text{monedas}[i] + \text{opt}(i+2, j) & \text{si } \text{monedas}[i+1] > \text{monedas}[j] \\ \text{monedas}[i] + \text{opt}(i+1, j-1) & \text{sino} \end{cases}$$

$$\text{moneda}_j + \text{opt}_j = \begin{cases} \text{monedas}[j] + \text{opt}(i+1, j-1) & \text{si } \text{monedas}[i] > \text{monedas}[j-1] \\ \text{monedas}[j] + \text{opt}(i, j-2) & \text{sino} \end{cases}$$

2.2. Algoritmo para obtener la reconstrucción de la solución

Partiendo de la matriz óptima, desarrollamos un algoritmo que brinde la secuencia de monedas que se deberían elegir para obtener la ganancia máxima, mediante llamadas recursivas.

1. Casos base.

- $i < j$ El rango de monedas es inválido, finalizo la recursión y devuelvo la solución.
- $i = j$ Solo queda una moneda por elegir, se agrega a la solución y se devuelve el conjunto.

2. Evaluación de alternativas (esta es la parte donde analizamos que sub problema conforma la solución del problema actual y se simula la elección de monedas).

- Se calcula opt_i y opt_j de la misma manera que en el algoritmo anterior, siendo los óptimos si se elige la moneda *i* o la moneda *j*, y en consecuencia que moneda elegiría Mateo (la mayor), y como estas decisiones afectan las dimensiones de nuestro problema.
- Analizamos que óptimo nos da la mayor ganancia si le sumamos la moneda correspondiente (porque es lo que el algoritmo hubiera hecho), y en base al mayor agregamos el índice *i* o el *j* a la solución, respectivamente.
- Por último vemos que moneda elige Mateo, cual es la mayor, y llamamos a la función recursivamente con el nuevo rango *i*, *j* conseguido luego de analizar las decisiones que tomarían ambos bandos.

```

1 def solucion(opt, monedas, s, i, j):
2     if i > j:
3         return s
4
5     if i == j:
6         s.append(i)
7         return s
8
9     opt_i = opt[i + 2][j] if monedas[i + 1] > monedas[j] else opt[i +
10    1][j - 1]
11    opt_j = opt[i + 1][j - 1] if monedas[i] > monedas[j - 1] else opt[i
12    ][j - 2]
13
14    if monedas[i] + opt_i > monedas[j] + opt_j:
15        s.append(i)
16
17        if monedas[i + 1] > monedas[j]:
18            return solucion(opt, monedas, s, i + 2, j)
19        else:
20            return solucion(opt, monedas, s, i + 1, j - 1)
21    else:
22        s.append(j)
23
24        if monedas[i] > monedas[j - 1]:
25            return solucion(opt, monedas, s, i + 1, j - 1)
26        else:
27            return solucion(opt, monedas, s, i, j - 2)

```

Siguendo estos pasos recursivamente obtenemos la secuencia de monedas que Sophia debería elegir para obtener la ganancia máxima.

3. Complejidad

La complejidad del algoritmo se puede ver como la complejidad de las siguientes partes, tomando siempre N , como la cantidad de monedas.

3.1. Complejidad del algoritmo para obtener la matriz optima

El algoritmo para obtener la matriz optima, tiene la siguiente complejidad.

1. Crear la matriz **opt** de $N \times N$ $\mathcal{O}(N^2)$.
2. Plantear los casos base de una moneda $\mathcal{O}(N)$.
3. Plantear los casos base de dos monedas $\mathcal{O}(N)$.
4. Completar la matriz optima $\mathcal{O}(N^2)$, recorro la matriz de $N \times N$ y hago un trabajo de $\mathcal{O}(1)$ por iteracion.

Por lo que la complejidad nos queda.

$$T(N) = \mathcal{O}(N^2 + N + N + N^2) = \mathcal{O}(N^2)$$

3.2. El algoritmo para reconstruir la solución

El algoritmo para reconstruir la solución tiene la siguiente complejidad.

1. Trabajo por llamada recursiva. En cada llamada realiza operaciones como comparaciones, accesos a listas y operaciones aritméticas, todas con una complejidad de $\mathcal{O}(1)$.
2. Número de llamadas recursivas. En cada llamada realiza un llamado recursivo, que achica o la variable i , o la j , en una o dos unidades. Como el rango inicial es de N , a lo sumo tendremos N llamadas recursivas.

Por lo tanto la complejidad nos queda.

$$T(N) = \mathcal{O}(N)$$

3.3. Complejidad total

Finalmente la complejidad total del algoritmo, desde calcular la matriz de resultados, hasta calcular la secuencia que se debe seguir para llegar al máximo encriptado, es la suma de ambas partes del algoritmo. Por lo que la complejidad total queda.

$$T(N) = \mathcal{O}(N^2 + N) = \mathcal{O}(N^2)$$

4. Seguimiento de ejemplos

5. Ajuste por cuadrados mínimos

5.1. Hipotesis

Para verificar que el algoritmo tiene un tiempo de ejecución de $\mathcal{O}(N^2)$, realizamos un ajuste de cuadrados con la siguiente función.

$$F(N) = a * N^2 + b$$

donde a y b , son los parámetros a ajustar.

5.2. Datos de entrada

- N , el tamaño del problema (la cantidad de monedas)
- $T(N)$ el tiempo de ejecución para cada tamaño N

5.3. Recopilacion de datos

Para obtener mediciones confiables y reducir el ruido, seguimos los siguientes pasos:

- Múltiples tamaños. Probamos con 20 tamaños distintos, espaciados uniformemente entre $n = 100$ y $n = 1,000$ (podemos ver que a diferencia del tp anterior, ahora nuestro rango es mucho menor, esto ya que el algoritmo anterior era lineal y este polinomial).
- Múltiples datasets. Para cada tamaño n , generamos 10 conjuntos de datos aleatorios independientes
- Promedio de mediciones. El tiempo de ejecución reportado para cada tamaño n es el promedio aritmético de las mediciones obtenidas para ese tamaño.

5.4. Código usado

```
1 from util import time_algorithm
2 from TP2 import elegir
3
4 # Siempre seteamos la seed de aleatoridad para que los resultados
5   sean reproducibles
6 seed(12345)
7 np.random.seed(12345)
8
9 sns.set_theme()
10
11 def get_random_array(size: int):
12     return np.random.randint(0, 100.000, size)
13
14 def main():
15     # La variable x van a ser los valores del eje x de los graficos
16     # en todo el notebook
17     # Tamano minimo=100, tamano maximo=10kk, cantidad de puntos
18     # =20
19     x = np.linspace(100, 1000, 20).astype(int)
20
21     results = time_algorithm(elegir, x, lambda s: [get_random_array
22     (s)])
23
24     ax: plt.Axes
25     fig, ax = plt.subplots()
26     ax.plot(x, [results[i] for i in x], label="Medicion")
```



```

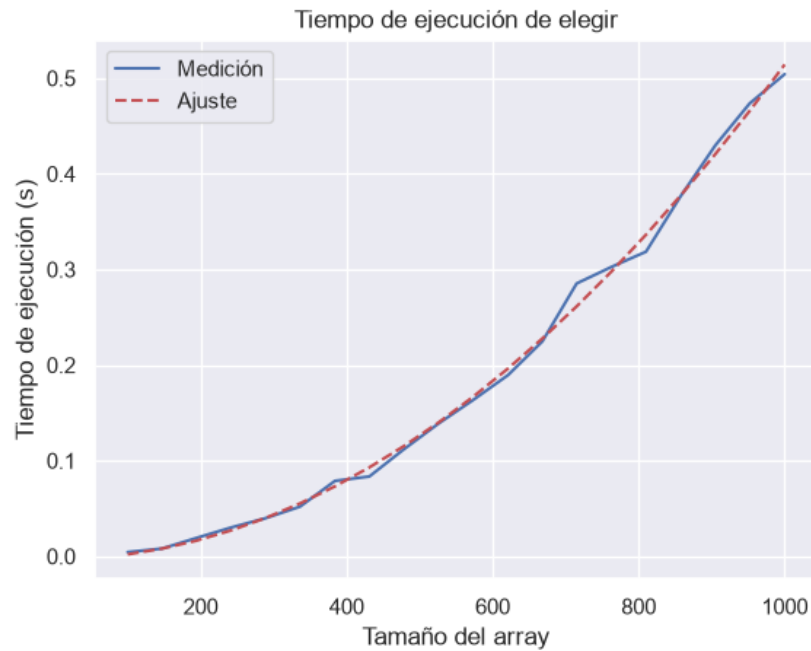
23 ax.set_title('Tiempo de ejecucion de elegir')
24 ax.set_xlabel('Tamano del array')
25 ax.set_ylabel('Tiempo de ejecucion (s)')
26
27 # scipy nos pide una funcion que recibe primero x y luego los
28 # parametros a ajustar:
29 f = lambda x, c1, c2: c1 * (x ** 2) + c2
30
31 c, pcov = sp.optimize.curve_fit(f, x, [results[n] for n in x])
32
33 print(f"c_1 = {c[0]}, c_2 = {c[1]}")
34 r = np.sum((c[0] * (x ** 2) + c[1] - [results[n] for n in x])
35 **2)
36 print(f"Error cuadratico total: {r}")
37
38 y_medido = np.array([results[n] for n in x])
39 y_predicho = c[0] * (x ** 2) + c[1]
40
41 ss_res = np.sum((y_medido - y_predicho)**2)
42 ss_tot = np.sum((y_medido - np.mean(y_medido))**2)
43
44 r_cuadrado = 1 - (ss_res / ss_tot)
45 print(f"R^2 = {r_cuadrado}")
46
47 ax.plot(x, [c[0] * (n ** 2) + c[1] for n in x], 'r--', label="
48 Ajuste")
49 ax.legend()
50
51 plt.show()
52
53 if __name__ == '__main__':
54     main()

```

5.5. Resultados

Los resultados del ajuste por cuadrado mínimo son.

- $a = 5,174732735449362e - 07$
- $b = -0,0031624210643063184$
- Error cuadrático total (R) = 0,001397511937062671
- Calidad del ajuste (R^2) = 0,9972484750254462



5.6. Interpretacion

Usamos el coeficiente de determinación R^2

- $R^2 = 1,00$: Ajuste perfecto
- R^2 mayor 0,98: Excelente ajuste $\rightarrow$ Complejidad verificada
- R^2 mayor 0,95: Muy buen ajuste $\rightarrow$ Complejidad muy probable
- R^2 menor 0,90: Ajuste pobre $\rightarrow$ Resultados no concluyentes

5.7. Conclusion

Con un valor de $R^2 = 0,99$ (mejor que el anterior), concluimos que la complejidad del algoritmo es $O(N^2)$, debido a la alta calidad del ajuste.

6. Conclusiones

Acá irían las conclusiones de todo nuestro trabajo :)